

Problem Set 3

CMSC 27200: Theory of Algorithms

Assigned January 24, Due January 31

Problem 0: Collaborators and Outside Sources

Who did you collaborate with?

Did you use any outside sources? If so, which sources did you use?

Problem 1: Recurrence Relations

Solve each of the following recurrence relations with the given initial value(s). Show your work.

- (a) $T(n) = T(\frac{n}{2}) + 3n, T(1) = 2$
- (b) $T(n) = 4T(\frac{n}{2}) + 3n^2, T(1) = 4$
- (c) $T(n) = 8T(\frac{n}{4}) + 2n + 7, T(1) = 1$
- (d) Extra credit: $T(n) = 6T(\frac{n}{2}) - 8T(\frac{n}{4}) + 2n + 3, T(1) = 1, T(2) = 1$

Problem 2: Local Minimum On A Complete Binary Tree (Problem 6 in Chapter 5 of Kleinberg-Tardos)

Let T be an n -node complete binary tree, where $n = 2^d - 1$ for some positive integer d . Each node v of T is labeled with a real number x_v , where the labels of the nodes are all distinct. A node v in T is called a *local minimum* if $x_v < x_w$, for all nodes w that are joined to v by an edge.

You are given such a complete binary tree, but the labeling $\{x_v\}_{v \in V}$ is given implicitly: for each node v , you can determine the value x_v by *probing* the node v .

- (a) Give an algorithm for finding a local minimum which only requires $O(\log n)$ queries. If you are using a recursive algorithm to search a subtree, what assumption(s) does your recursive algorithm make? If you are iteratively reducing to a smaller subtree, what invariant are you maintaining?

As described in the rubric, it is possible (and relatively easy) to earn an exemplary grade for this part.

- (b) Explain why your algorithm is correct. In other words, explain why you can narrow down the search area at each step.
- (c) Explain why your algorithm only requires $O(\log n)$ queries.

Problem 3: The Billy Problem

Acknowledgement: This problem is due to Andy Drucker who was inspired by The Family Circus.

Billy, a 7-year-old kid, is let loose in a park, which is an N -by- N grid of cells $(x, y) \in [N] \times [N]$. His starting location is the Southwest corner $(x, y) = (1, 1)$. He takes a walk in the park, always moving from a square to an adjacent square in one of the four cardinal directions. He never leaves the park and never visits the same square twice. Eventually he ends up napping somewhere in the park.

We are given an N -by- N matrix A , whose entries are

1. $A[i, j] = *$ if Billy never visits cell (i, j) .
2. $A[i, j] = \uparrow$ if Billy visited cell (i, j) and departed upwards to $(i, j + 1)$.
3. $A[i, j] = \leftarrow$ if Billy visited cell (i, j) and departed in the leftwards direction to $(i - 1, j)$.
4. $A[i, j] = \downarrow$ if Billy visited cell (i, j) and departed downwards to $(i, j - 1)$.
5. $A[i, j] = \rightarrow$ if Billy visited cell (i, j) and departed in the rightwards direction to $(i + 1, j)$.
6. $A[i, j] = BILLY$ if Billy is napping in cell (i, j) .

For example, $A[1, 1]$ will contain an arrow indicating the first step Billy took after entering the park (unless he stopped to nap immediately).

- (a) Give an algorithm for locating Billy which only requires $O(N)$ queries to the matrix A . If you are using a recursive algorithm to search a smaller area, what assumption(s) does your recursive algorithm make? If you are iteratively reducing the size of the area you are searching, what invariant are you maintaining?
[As described in the rubric, it is possible to earn an exemplary grade for this part.](#)
- (b) Explain why your algorithm is correct. In other words, explain why you can narrow down the search area at each step.
- (c) Explain why your algorithm only requires $O(N)$ queries.

Hint is available.

Problem 4: Implementing a List with an Array

Let's say that we want a list L which supports the following operations

Insert(x): Add an element x to the end of the list L .

L[i]: Returns the i th element in L .

Remove(i): Remove the i th element in L .

We can implement such a list L using an array A as follows:

1. To insert x at the end of the list, we add x to the next position of A .
2. To find the i th element of L , we find the index i' such that i' is the i th index of A which has not been removed. If we want to remove the i th element of L , we set $A[i'] = \text{Null}$.

Example 0.1. If we started with an empty list L and performed the operations *Insert(5)*, *Insert(7)*, *Insert(2)*, *Remove(2)*, *Insert(10)*, *Insert(15)*, *Insert(4)*, *Insert(9)*, *Remove(4)*, *Insert(12)*, *Remove(5)* then L would be $L = 5, 2, 10, 4, 12$ and the array A storing the elements of L would be

5	N	2	10	N	4	N	12
---	-----	---	----	-----	---	-----	----

where N represents a removed element which has been replaced by Null .

The challenge is how to find the index i' such that i' is the i th index of A which has not been removed.

- (a) Construct a data structure which allows you to find the index i' such that i' is the i th index of A which has not been removed in $O(\log(k + 2))$ time where k is the number of removals which have happened so far.

Hint is available.

Note: it is possible but very challenging to earn an exemplary grade for this part.

- (b) Explain why your data structure is correct.

Hints:

3. If you know the entries within distance two of a region, can you determine if Billy is napping in that region?

4. It may be helpful to store the indices of A which have been deleted in a binary tree where each node stores the size of the subtree rooted at that node. Given such a tree, how can we determine if i' is less than or greater than the root of the tree?

Note: Ordinary binary search trees are not guaranteed to have $O(\log n)$ runtime for their operations so a self-balancing tree like a red-black tree or a 2-3-4 tree would be needed to guarantee that the remove operations are done in $O(\log(k + 2))$ time. That said, unless you are aiming for an exemplary grade, you may ignore this issue and use an ordinary binary search tree.